

Reviewer Pack — Minimal Tensor Rank of Algebraic Curves over Function Fields...

Entry ea45b50069b0 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-24 14:24:15 UTC

Minimal Tensor Rank of Algebraic Curves over Function Fields vs Exponential Depth of Frege Proofs

Entry ID: ea45b50069b0 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-24 14:24:15 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry (Algebraic Curves) **Field B** (complexity object): Complexity Theory: Frege Proof Complexity

Statement:

For every Frege proof system P with exponential depth D , there exists an absolute constant c such that the minimal tensor rank of any algebraic curve defined over a function field F_q with q elements is at least $c * \log_2(q^D)$.

Rationale (proposer's reasoning):

Algebraic curves offer a rich source of invariants that can be computed in terms of algebraic operations. The exponential depth of Frege proofs provides a measure of the complexity of proving tautologies. A connection between these two areas could reveal deep structural properties related to proof complexity.

Taxonomy category: ALGEBRAIC_GEOMETRY × COMPLEXITY_THEORY (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 0ad0012fda833572

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the mean of the ratio of minimal tensor rank to $\log_2(q^D)$ across all seeds exceeds an absolute constant c , where no seed produces a ratio exceeding 10.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 6 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "algebraic geometry" AND "tensor rank" AND Frege proof complexity" - "exponential depth" IN title AND "minimal tensor rank" AND algebraic curves" - "Frege proof system" AND "exponential depth" AND function fields AND tensor rank

Top relevant hits considered: - [http://arxiv.org/abs/2004.01492v2] Tensor Rank and Complexity - [http://arxiv.org/abs/2605.21784v1] Constructions of Rank-Metric Codes of Small Tensor Rank - [http://arxiv.org/abs/2103.07411v2] A Normal Form Algorithm for Tensor Rank Decomposition - [http://arxiv.org/abs/2503.14756v3] SceneEval: Evaluating Semantic Coherence in Text-Conditioned 3D Indoor Scene Synthesis - [http://arxiv.org/abs/2408.02211v2] SceneMotifCoder: Example-driven Visual Program Learning for Generating 3D Object Arrangements - [http://arxiv.org/abs/2509.13291v1] Towards an Embodied Composition Framework for Organizing Immersive Computational Notebooks

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=0, elapsed=0.0s

5.1 Generated Python source

```
import random
from fractions import Fraction
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    # Generate a random function field F_q with q elements
    q = 2 ** random.randint(1, 5) # q is a power of 2 for simplicity
    F_q = {i for i in range(q)}

    # Define an algebraic curve over F_q (example: y^2 = x^3 + x + 1)
    def algebraic_curve(x):
        return (x ** 3 + x + 1) % q

    # Construct a Frege proof for polynomially sized circuits computing XOR tautologies
    n = random.randint(5, 40) # Number of variables in the XOR circuit
    D = 2 * n # Exponential depth of the Frege proof (simplified)

    # Compute the minimal tensor rank of the algebraic curve
    tensor_rank = len(F_q) # Simplified for demonstration

    # Calculate the ratio of minimal tensor rank to log_2(q^D)
    if q ** D == 0:
        return {
```

```

        "metric_name": "ratio",
        "metric_value": float('inf'),
        "instances_tested": 1,
        "conjecture_holds": False,
        "counterexample": "q**D is zero"
    }

ratio = Fraction(tensor_rank, q ** D).limit_denominator()
log_ratio = math.log2(q ** D)
if log_ratio == 0:
    return {
        "metric_name": "ratio",
        "metric_value": float('inf'),
        "instances_tested": 1,
        "conjecture_holds": False,
        "counterexample": "log_2(q**D) is zero"
    }

ratio_value = ratio / log_ratio

return {
    "metric_name": "ratio",
    "metric_value": float(ratio_value),
    "instances_tested": 1,
    "conjecture_holds": ratio_value > 0.5, # Simplified threshold for demonstration
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i + 1 for i in range(30)] # Default to first 30 primes

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_ratio = sum(result["metric_value"] for result in results) / len(results)
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_ratio} std=0.0 support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_ratio} std=0.0 support_fraction={support_fraction}")
    else:
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"first failing seed\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
sample': ''}
```

```

TRIAL: {'metric_name': 'ratio', 'metric_value': 0.0, 'instances_tested': 1, 'conjecture_holds': False}
TRIAL: {'metric_name': 'ratio', 'metric_value': 0.0, 'instances_tested': 1, 'conjecture_holds': False}
TRIAL: {'metric_name': 'ratio', 'metric_value': 0.0, 'instances_tested': 1, 'conjecture_holds': False}
TRIAL: {'metric_name': 'ratio', 'metric_value': 0.0, 'instances_tested': 1, 'conjecture_holds': False}
TRIAL: {'metric_name': 'ratio', 'metric_value': 0.0, 'instances_tested': 1, 'conjecture_holds': False}
TRIAL: {'metric_name': 'ratio', 'metric_value': 0.0, 'instances_tested': 1, 'conjecture_holds': False}
TRIAL: {'metric_name': 'ratio', 'metric_value': 4.238552517361111e-07, 'instances_tested': 1, 'conjecture_holds': False}
TRIAL: {'metric_name': 'ratio', 'metric_value': 0.0, 'instances_tested': 1, 'conjecture_holds': False}
TRIAL: {'metric_name': 'ratio', 'metric_value': 0.0, 'instances_tested': 1, 'conjecture_holds': False}
TRIAL: {'metric_name': 'ratio', 'metric_value': 4.238552517361111e-07, 'instances_tested': 1, 'conjecture_holds': False}
TRIAL: {'metric_name': 'ratio', 'metric_value': 0.0, 'instances_tested': 1, 'conjecture_holds': False}
TRIAL: {'metric_name': 'ratio', 'metric_value': 0.0, 'instances_tested': 1, 'conjecture_holds': False}
TRIAL: {'metric_name': 'ratio', 'metric_value': 0.0, 'instances_tested': 1, 'conjecture_holds': False}
TRIAL: {'metric_name': 'ratio', 'metric_value': 0.0, 'instances_tested': 1, 'conjecture_holds': False}
TRIAL: {'metric_name': 'ratio', 'metric_value': 0.0, 'instances_tested': 1, 'conjecture_holds': False}
TRIAL: {'metric_name': 'ratio', 'metric_value': 0.0, 'instances_tested': 1, 'conjecture_holds': False}
RESULT: FALSIFIED counterexample="first failing seed" first_failing_seed=11

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

The empirical test only includes a single instance (seed) before the conjecture was falsified, which is insufficient to draw conclusions about the general case. This could be due to n too small, where ' n ' refers to the number of instances tested. The metric does not scale trivially with n , as it depends on the logarithm of q^D .

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Safety rail: critic_challenge_falsified | original: The test results indicate that for at least one seed (first failing seed=11), the conjecture does not hold, as the mean ratio is less than or equal to | next: Further investigation is needed to determine if this counterexample represents a general trend or if it is an anomaly. Additional testing with more seeds and larger values of ' n ' should be conducted.

11. Audit log (LLM calls)

Total LLM calls: 11

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12187
2	propose	ollama_remote	glm4:latest	0	0	9036
3	preregistration	ollama_remote	glm4:latest	0	0	5291
4	novelty	ollama_remote	glm4:latest	0	0	4697
5	novelty	ollama_remote	glm4:latest	0	0	5823
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9595
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8541
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7503
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8777
10	critic	ollama_remote	glm4:latest	0	0	14134
11	judge	ollama_remote	glm4:latest	0	0	7768

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 93354 ms total latency. Provider mix: {'ollama_remote': 11}

(full prompt+response transcripts available in research/audit/ea45b50069b0.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/ea45b50069b0.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/ea45b50069b0.tar.gz (if generated)